

Tài liệu ôn phỏng vấn - Project Oder

Dành cho project đặt món/nhà hàng dùng React, Express, MongoDB/Mongoose.

Mục tiêu: giúp bạn ôn kiến thức, chuẩn bị câu trả lời khi bảo vệ đồ án hoặc đi phỏng vấn fresher/intern full-stack.

1. Tóm tắt project để giới thiệu khi phỏng vấn

Gợi ý giới thiệu ngắn: Project Oder là hệ thống đặt món ăn gồm client và admin. Client có xem sản phẩm, tìm kiếm, xem chi tiết, giỏ hàng, checkout, đặt bàn hoặc giao hàng. Admin có đăng nhập, quản lý sản phẩm, danh mục, tài khoản, phân quyền, đơn hàng, nhà hàng, quảng cáo và backup sản phẩm. Backend dùng Express, MongoDB/Mongoose, cookie để lưu token đăng nhập và cartId. Frontend dùng React, React Router, fetch API, Bootstrap và một số thư viện hỗ trợ PDF, chart, editor.

Công nghệ chính

- **Frontend:** React 19, React Router DOM, Bootstrap, Recharts, TinyMCE, jsPDF/pdfmake, Context API cho giỏ hàng.
- **Backend:** Node.js, Express 5, Mongoose, cookie-parser, cors, multer, nodemailer/resend.
- **Database:** MongoDB, thiết kế collection cho product, cart, order, account, role, user, restaurant, table.
- **Authentication:** admin login bằng email/password, token lưu cookie, middleware kiểm tra cookie.
- **Authorization:** role/permission lưu trong collection decentralization.

2. Câu hỏi phỏng vấn tổng quan về project

1. Project của em giải quyết bài toán gì?

Project giải quyết bài toán đặt món trực tuyến cho nhà hàng/quán ăn. Người dùng có thể xem món, tìm kiếm, thêm vào giỏ, đặt bàn hoặc đặt giao hàng. Phía admin quản lý sản phẩm, danh mục, đơn hàng, tài khoản, vai trò và nhà hàng.

2. Vì sao em tách frontend và backend?

Tách frontend/backend giúp chia rõ trách nhiệm. Frontend tập trung giao diện và trải nghiệm người dùng, backend xử lý nghiệp vụ, xác thực, dữ liệu và API. Cách này dễ deploy riêng, dễ bảo trì và có thể mở rộng mobile app sau này dùng chung API.

3. Kiến trúc luồng request trong project như thế nào?

React gọi API bằng fetch, gửi cookie kèm request với `credentials: "include"`. Express nhận request, đi qua middleware như CORS, JSON parser, cookie parser, auth middleware nếu là admin, sau đó route chuyển đến controller. Controller xử lý nghiệp vụ với Mongoose model và trả JSON về frontend.

4. Em đã làm chức năng nào nổi bật nhất?

Nên chọn 2-3 chức năng bạn nắm chắc: đăng nhập admin bằng cookie token, quản lý sản phẩm có tìm kiếm/lọc/phân trang/sort, giỏ hàng dùng cookie cartId, checkout có đặt bàn và kiểm tra bàn trống, hoặc phân quyền theo role.

5. Nếu có thêm thời gian em sẽ cải thiện gì?

Cải thiện bảo mật password từ MD5 sang bcrypt, token sang JWT/session an toàn hơn, thêm validate đầy đủ, thêm test, chuẩn hóa response API, thêm index MongoDB cho trường tìm kiếm, xử lý lỗi tập trung, và dùng transaction cho luồng checkout để tránh lỗi khi đặt bàn/đơn hàng.

3. Backend: Express, Route, Middleware, Controller

6. Express trong project dùng để làm gì?

Express tạo HTTP server và định nghĩa API. Trong project, backend dùng `app.use(express.json())` để đọc JSON body, `cookieParser()` để đọc cookie, `cors()` để cho frontend gọi API, rồi mount route client và admin.

7. Route và controller khác nhau thế nào?

Route định nghĩa endpoint và HTTP method, ví dụ GET/POST/PATCH/DELETE. Controller chứa logic xử lý chính: đọc req, kiểm tra dữ liệu, gọi database qua model, rồi trả response.

8. Middleware là gì? Project dùng middleware ở đâu?

Middleware là hàm chạy giữa request và controller. Project dùng middleware xác thực admin: đọc `token` từ cookie, tìm account trong database, lấy role và lưu vào `res.locals` để controller/route phía sau sử dụng.

9. Vì sao admin route cần `requireAuth` ?

Vì các API admin như quản lý sản phẩm, tài khoản, đơn hàng là dữ liệu nhạy cảm. Middleware đảm bảo chỉ người đã đăng nhập và có token hợp lệ mới được truy cập.

10. CORS trong project có vai trò gì?

CORS cho phép frontend ở domain khác, ví dụ `http://localhost:3000` hoặc domain Vercel, gọi API backend. Vì project dùng cookie nên CORS cần bật `credentials: true`, frontend cũng phải gửi `credentials: "include"`.

4. Authentication và Authorization

11. Luồng đăng nhập admin hoạt động thế nào?

Frontend gửi email/password lên API login. Backend tìm account theo email, kiểm tra trạng thái tài khoản, so sánh password đã hash MD5, nếu hợp lệ thì set cookie `token`. Các request admin sau đó gửi cookie này để middleware xác thực.

12. Authentication khác Authorization thế nào?

Authentication là xác minh người dùng là ai, ví dụ login bằng email/password. Authorization là xác định người đó được làm gì, ví dụ role admin có quyền thêm/sửa/xóa sản phẩm còn nhân viên chỉ xem đơn hàng.

13. Điểm yếu bảo mật hiện tại là gì?

MD5 không còn an toàn cho password vì quá nhanh và dễ bị dò bằng rainbow table. Nên dùng bcrypt/argon2 có salt và cost factor. Ngoài ra nên set cookie `httpOnly`, `secure`, `sameSite`, thêm giới hạn login sai và chuẩn hóa kiểm tra permission.

14. Token lưu trong cookie có ưu/nhược điểm gì?

Ưu điểm là trình duyệt tự gửi cookie theo request, phù hợp với session. Nhược điểm là cần cấu hình CORS/credentials đúng và phải phòng CSRF nếu cookie tự động gửi. Nếu dùng JWT trong header thì tránh CSRF hơn nhưng cần quản lý lưu trữ token ở frontend.

5. MongoDB và Mongoose

15. Vì sao project dùng Mongoose?

Mongoose giúp định nghĩa schema, model, kiểu dữ liệu, default value, timestamps và thao tác MongoDB bằng API rõ ràng hơn. Ví dụ Product có name, price, stock, category, status, deleted, slug và restaurant_id.

16. Collection Product có những trường quan trọng nào?

Product gồm `name`, `price`, `img`, `stock`, `category`, `status`, `deleted`, `slug`, `description`, `restaurant_id`, `position`, `createdAt`, `updatedAt`.

17. Soft delete là gì? Project dùng thế nào?

Soft delete là không xóa bản ghi khỏi database mà đánh dấu `deleted: true` và lưu `deletedAt`. Khi query danh sách sản phẩm, backend lọc `deleted: false`. Cách này giúp khôi phục hoặc audit dữ liệu sau này.

18. Khi nào nên dùng populate?

Khi document lưu ObjectId tham chiếu sang collection khác, ví dụ `restaurant_id` tham chiếu Restaurant. Dùng populate để lấy thông tin nhà hàng kèm sản phẩm thay vì chỉ lấy id.

19. Index database nên thêm ở đâu?

Nên thêm index cho các trường hay lọc/tìm kiếm như `name`, `status`, `category`, `deleted`, `restaurant_id`, `createdAt`. Với search text có thể dùng text index hoặc regex có giới hạn.

6. Sản phẩm, tìm kiếm, lọc, phân trang

20. API danh sách sản phẩm admin xử lý những gì?

API tạo object query gồm `deleted: false`, thêm status nếu có, thêm regex name nếu search, thêm category nếu có. Sau đó đếm tổng sản phẩm, tính phân trang, sort theo query hoặc mặc định position desc, rồi dùng limit/skip để trả dữ liệu.

21. Phân trang bằng limit/skip hoạt động thế nào?

`limit` giới hạn số item mỗi trang, `skip` bỏ qua số item của các trang trước. Ví dụ page 3, limit 10 thì skip 20. Cách này đơn giản nhưng với dữ liệu rất lớn có thể chậm, lúc đó nên dùng cursor pagination.

22. Regex search có rủi ro gì?

Regex tiện cho tìm kiếm gần đúng nhưng nếu không kiểm soát input có thể gây query chậm. Nên escape ký tự đặc biệt, giới hạn độ dài keyword và cân nhắc text index.

23. Vì sao cần validate khi tạo/sửa sản phẩm?

Validate giúp tránh dữ liệu sai như giá âm, stock không phải số, tên rỗng, category không hợp lệ. Backend vẫn phải validate kể cả frontend đã kiểm tra vì client có thể bị bypass.

7. Giỏ hàng và checkout

24. Giỏ hàng trong project được lưu như thế nào?

Backend tạo hoặc tìm Cart theo cookie `cartId`. Nếu chưa có `cartId`, tạo cart mới, lưu cookie 14 ngày. Cart chứa danh sách `product_id` và `quantity`, có thể gán `restaurant_id` để đảm bảo giỏ chỉ chứa món của một nhà hàng.

25. Vì sao giỏ hàng giới hạn một nhà hàng?

Trong đặt món nhà hàng, một đơn thường thuộc về một nhà hàng để xử lý bếp, bàn, giao hàng và thanh toán. Nếu giỏ chứa nhiều nhà hàng, đơn hàng phải tách nhiều order, logic phức tạp hơn.

26. Luồng thêm sản phẩm vào giỏ hoạt động thế nào?

Backend kiểm tra sản phẩm tồn tại và chưa bị xóa. Nếu cart đã có `restaurant_id` khác với sản phẩm mới thì trả lỗi 409. Nếu sản phẩm đã có trong cart thì cộng `quantity`, nếu chưa có thì push item mới vào mảng `products`.

27. Luồng checkout hoạt động thế nào?

Backend đọc `cartId` từ cookie, kiểm tra thông tin người đặt, kiểm tra `orderType`. Nếu dine-in thì phải chọn bàn. Backend lấy sản phẩm trong cart, tạo order, nếu đặt bàn thì tìm bàn available và đổi sang occupied, sau đó save order và xóa sản phẩm trong cart.

28. Rủi ro trong checkout là gì?

Rủi ro lớn là tạo order thành công nhưng update bàn hoặc clear cart lỗi, hoặc hai người cùng đặt một bàn. Project đã dùng `findOneAndUpdate` với điều kiện `status: "available"` để giảm race condition. Tốt hơn nữa là dùng MongoDB transaction cho toàn bộ luồng.

8. React frontend

29. React Router trong project dùng để làm gì?

React Router chia giao diện theo path: `/admin/*` cho admin layout, `/admin/auth/*` cho login admin, `/restaurant-owner/*` cho chủ nhà hàng, `/*` cho client và `/user/auth/*` cho login/register user.

30. Layout component có lợi ích gì?

Layout giúp tái sử dụng phần chung như header, sidebar, footer. AdminLayout dùng cho các trang quản trị, DefaultLayout dùng cho client, RestaurantOwnerLayout dùng cho chủ nhà hàng.

31. Vì sao apiFetch tự động xử lý 401/403?

Để gom logic gọi API vào một chỗ. Khi backend trả 401 hoặc 403, frontend có thể xử lý tập trung như chuyển về login hoặc hiển thị không có quyền, thay vì lặp code ở từng component.

32. Context API trong giỏ hàng dùng khi nào?

Context API phù hợp khi nhiều component cần dùng chung trạng thái giỏ hàng như header mini cart, trang cart, nút add to cart. Nó tránh truyền props qua nhiều cấp.

9. Câu hỏi về phân quyền và admin

33. Role-based permission là gì?

Là cơ chế gán quyền theo vai trò. Ví dụ role Admin có toàn quyền, role Staff chỉ xem/cập nhật đơn, role Restaurant Owner chỉ quản lý sản phẩm/đơn của nhà hàng mình.

34. Phân quyền nên kiểm tra ở frontend hay backend?

Frontend chỉ dùng để ẩn/hiện nút cho trải nghiệm tốt hơn. Backend mới là nơi bắt buộc kiểm tra quyền vì người dùng có thể gọi API trực tiếp bằng Postman/devtools.

35. Nếu user không có quyền xóa sản phẩm thì backend nên trả gì?

Nên trả HTTP 403 Forbidden với message rõ ràng. 401 dùng cho chưa đăng nhập hoặc token không hợp lệ, còn 403 dùng cho đã đăng nhập nhưng không đủ quyền.

10. Câu hỏi JavaScript/Node.js hay gặp

36. async/await dùng để làm gì?

Dùng để viết code bất đồng bộ dễ đọc hơn Promise chain. Trong backend, gọi MongoDB là async nên controller dùng `await Product.find(...)` để chờ kết quả trước khi trả response.

37. Promise là gì?

Promise đại diện cho một tác vụ bất đồng bộ có thể pending, fulfilled hoặc rejected. Các hàm database, fetch API thường trả về Promise.

38. Sự khác nhau giữa == và === ?

`==` so sánh có ép kiểu, `===` so sánh cả giá trị và kiểu dữ liệu. Nên ưu tiên `===` để tránh lỗi khó đoán.

39. map, filter, reduce dùng thế nào?

`map` biến đổi mảng, `filter` lọc phần tử, `reduce` gom mảng thành một giá trị. Ví dụ tính tổng giỏ hàng có thể dùng `reduce`: `tổng price * quantity`.

40. REST API là gì?

REST là cách thiết kế API dựa trên resource và HTTP method: GET lấy dữ liệu, POST tạo mới, PATCH/PUT cập

nhật, DELETE xóa. Project dùng REST cho sản phẩm, cart, checkout, account, role.

11. Câu hỏi tình huống khi bảo vệ đồ án

41. Nếu admin xóa nhầm sản phẩm thì sao?

Vì project dùng soft delete, sản phẩm chỉ bị đánh dấu deleted chứ chưa xóa vĩnh viễn. Có thể xây thêm chức năng khôi phục từ danh sách backup/thùng rác.

42. Nếu mất kết nối MongoDB thì hệ thống xử lý ra sao?

Hiện tại backend log lỗi kết nối. Khi nâng cấp, nên dừng server nếu không kết nối được DB, thêm retry, health check và trả lỗi chuẩn khi API không truy cập được database.

43. Nếu hai khách cùng đặt một bàn thì sao?

Backend dùng điều kiện update bàn chỉ khi status là available. Request nào update trước sẽ chiếm bàn, request sau không tìm thấy bàn available và trả lỗi yêu cầu chọn bàn khác.

44. Nếu giỏ hàng có sản phẩm đã bị xóa thì checkout thế nào?

Backend khi checkout lấy thông tin từng sản phẩm. Nếu không tìm thấy sản phẩm thì bỏ qua. Nếu sau khi lọc không còn sản phẩm hợp lệ thì trả lỗi. Có thể cải thiện bằng cách báo rõ sản phẩm nào không còn bán.

45. Làm sao để deploy project?

Frontend có thể deploy lên Vercel. Backend deploy lên Render/Railway/VPS. Cần cấu hình biến môi trường như `MONGODB_URI`, domain CORS, cookie secure/sameSite phù hợp môi trường production.

12. Kiến thức cần ôn theo mức ưu tiên

Ưu tiên cao

- HTTP method, status code 200/201/400/401/403/404/409/500.
- Express route, controller, middleware.
- Cookie, CORS, credentials, authentication, authorization.
- Mongoose schema, model, query, updateOne, updateMany, findOneAndUpdate.
- React component, props, state, hooks, routing, layout.
- Luồng giỏ hàng và checkout của chính project.

Ưu tiên trung bình

- Phân trang, sort, filter, search.
- Soft delete và backup dữ liệu.
- Validate dữ liệu frontend/backend.
- Thiết kế collection MongoDB và quan hệ bằng ObjectId.
- Xử lý lỗi tập trung và chuẩn hóa response JSON.

Ưu tiên cải thiện để ghi điểm

- Thay MD5 bằng bcrypt.
- Thêm HTTP-only secure cookie hoặc JWT strategy rõ ràng.
- Thêm kiểm tra permission ở backend cho từng API.
- Dùng transaction cho checkout.
- Thêm unit/integration test cho auth, product, cart, checkout.
- Thêm index MongoDB và tối ưu search.

13. Checklist ôn trước ngày phỏng vấn

1. Chạy được backend và frontend local, biết port và lệnh chạy.
2. Demo được login admin, danh sách sản phẩm, thêm/sửa/xóa sản phẩm.
3. Demo được client thêm giỏ hàng và checkout.
4. Giải thích được schema Product, Cart, Order, Account, Role.
5. Giải thích được cookie token và cookie cartId.
6. Nêu được ít nhất 5 điểm hạn chế và cách cải thiện.
7. Chuẩn bị câu trả lời 1 phút: “Em đã học được gì từ project này?”.

14. Câu trả lời mẫu: Em học được gì từ project?

Qua project này em học được cách xây dựng một ứng dụng full-stack từ frontend đến backend: thiết kế route React, gọi API, xử lý cookie, xây dựng API Express, dùng Mongoose làm việc với MongoDB, tổ chức controller/model/route, xử lý giỏ hàng, checkout và phân quyền admin. Em cũng hiểu rõ hơn về các vấn đề thực tế như validate dữ liệu, bảo mật đăng nhập, CORS khi dùng cookie, soft delete, phân trang, tìm kiếm và cách cải thiện hệ thống để gần production hơn.